This work is licensed under a Creative Commons license

Attribution-NonCommercial-NoDerivatives 4.0 International
(CC BY-NC-ND 4.0)

You are free to:

- **Share** – copy and redistribute the material in any medium or format.

Under the following terms:

- **Attribution** – You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- **NonCommercial** – You may not use the material for commercial purposes.
- **NoDerivatives** – If you remix, transform, or build upon the material, you may not distribute the modified material.

Software Reverse Engineering

Part I – Foundations

Simone Aonzo, PhD

Last update: 2026-02-17T15:03:36

Outline

Reverse Engineering (RE)

Reverse Engineering (RE) is the process of understanding how an existing system or object works when the original design details are unavailable or incomplete.

It relies on *deductive reasoning*: drawing conclusions from one or more general premises assumed to be true (e.g., observed facts).

Why? Historically driven by economic incentives and military needs 😔

Where? Where an engineering process has produced "something"
Computing, mechanics, electronics, chemistry, biology, ...

How and what?

Reverse engineering is typically iterative and may include:

1. Information Gathering
2. Structural (Static) Analysis
3. Behavioral (Dynamic) Analysis
4. Abstract Modeling
5. Hypothesis Validation

Types of Analysis:

- Black-Box
- Gray-Box ← Binary Analysis
- White-Box...? Blurred boundary

RE: implementation → specification

Reverse engineering is the reconstruction of higher-level abstractions from lower-level artifacts, independent of how much internal access is available.

Scope of RE in Computer Science and Engineering

RE applied to:

- Hardware
- Software
- Network Protocols

Motivations:

- Digital forensics (e.g., **malware analysis**)
- Legacy system updates
- Interoperability
- Security auditing
- Vulnerability discovery
- New features (e.g., APIs for “botting”)
- . . .
- Theft of intellectual property
- Cracking (i.e., breaking copy protection)
- Cheating

Software Reverse Engineering (SRE)

SRE is the process of analyzing an application to reconstruct its design, architecture, algorithms, and source code (or high-level representations) without having access to the original documentation or source code.

In this course...

We deal with **SRE** to understand how malicious software, i.e., **malware**, works by analyzing it in a **grey-box** fashion

SRE Challenges

- Low-level languages
- Often, no “symbols” (i.e., meaningful names)
- Almost no type information
- No class/module/namespace/library boundaries
- Code and data can be mixed – and they usually do
- Difficulty in adding/changing/removing instructions
- ...

With some effort and the right tools, we'll break programs apart to learn how they work and what they do — an old saying goes:

“Once you speak machine code, then every program becomes open-source” 😆

Legal Considerations

Different countries $\implies$ different laws

Directive 2009/24/EC of the *European Union*

- **Article 5** grants any legitimate user the right "to observe, study or test the functioning" of a program [1] by loading, running, transmitting or storing it, in order to determine the ideas and principles underlying its elements. Contractual terms attempting to prohibit such acts are null and void.
- **Article 6** permits decompilation (i.e., conversion of machine code into a human-readable form) solely where necessary for interoperability, and only if the user already holds a license and the interoperability information is not otherwise available.

$\implies$ Limited SRE aimed at interoperability is lawful in the EU, under strict statutory conditions that cannot be overridden by contract

[1] Author's Note: a malware is a program!

Outline

Reality check

Today, there are hundreds of programming languages...
low or high-level...
which follow a dozen paradigms...
but in the end, they all have something in common:

None of them are the code that is actually executed by the CPU!

Machine Code

A binary sequence which instructs the CPU on what operations to perform
$\implies$ different CPU architectures have different machine code

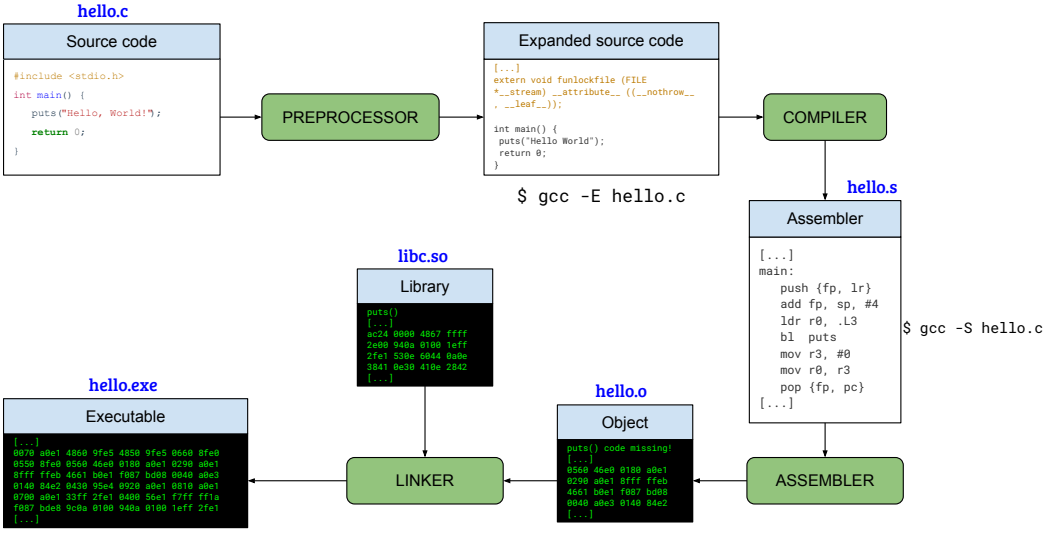
How software is born

1. Compiled: Ahead-Of-Time (AOT) into machine code
   - Source code →
   - Intermediate Representation (IR) →
   - Assembly code →
   - Machine code →
   - CPU
2. Interpreted: needs a virtual machine
   - Source code →
   - Bytecode →
   - Interpreter →
   - Virtual Machine (VM) →
   - CPU

Just-In-Time (JIT) Compilation

Modern interpreters compile optimized Bytecode at runtime

E.g., a compiled language: C



Executable File

It is a structured file that includes machine code instructions, metadata, and loader directives.

These allow an operating system to load the file into memory and run it directly on a specific CPU architecture.

The most common executable file format are:

- *Portable Executable* (PE) on Microsoft Windows
- *Mach Object* (Mach-O) on macOS and iOS
- *Executable and Linkable Format* (ELF) on Linux and Android

Different programming languages ⇒ different executables

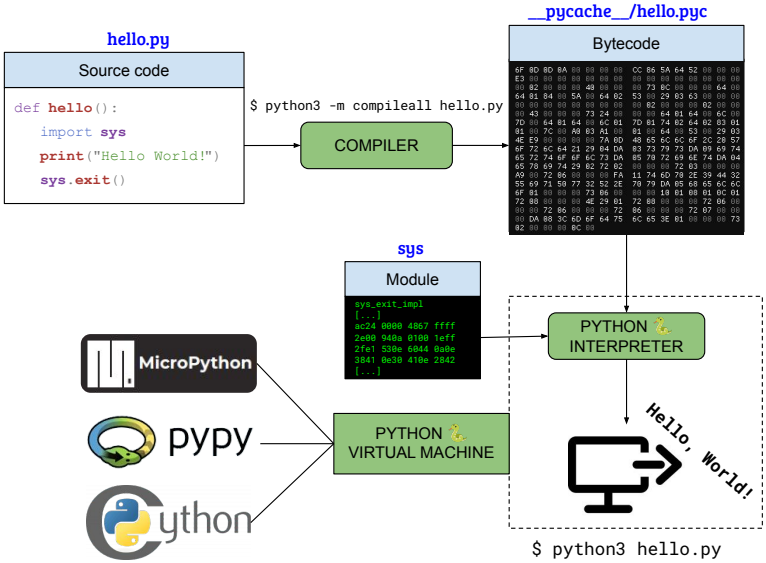
Even if two programs are *semantically equivalent* (do the same thing):

- The generated assembly|executable can differ significantly
- Languages expose different **language abstractions**
  - C++: RAII, templates, virtual methods → ctors/dtors, vtables, code duplication
  - Rust: ownership/borrowing, traits, pattern matching → move checks, monomorphization
- Languages rely on different **runtime environments**
  - C++: exception unwinding, RTTI, `libstdc++`/`libc++`
  - Rust: panic strategy (abort vs unwind), `std` runtime, custom allocators

To give you a rough idea:

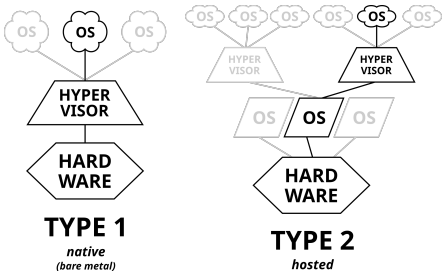
Compiling a "Hello World" program (without functions) in C, C++, or Rust generates an executable with 25, 80, or 564 *functions*, respectively

E.g., an interpreted language: Python



Virtual Machines – Once and for all!

1. **Interpreter** – Virtualize software execution to run programs
   - Language-defined bytecode (e.g., CPython and JavaVirtualMachine)
   - Custom bytecode (e.g., VM obfuscation)
2. System – Virtualize hardware resources to run entire OSes
   - **Emulator**: fully simulates hardware (slow, cross-arch)
     - E.g., mGBA (Game Boy), PCSX2 and RPCS3 (PlayStation 2 and 3)
     - E.g., QEMU, but it can also use hardware-assisted virtualization 👇
   - **Hypervisor**: shares real hardware (fast, near-native)
     - Type 1: native (e.g., Hyper-V, Xen, VMware ESXi)
     - Type 2: hosted (e.g., VirtualBox, VMware Workstation)



Outline

Historical Context (1930s)

- **Alonzo Church**: λ-calculus as a formal model of computation
- **Alan Turing**: Turing machine as another formal model
- Both turned out to describe the same class of *computable functions*
- Functions for which there exists an algorithm (a finite set of rules) that can be followed to produce the correct output for any valid input in a finite amount of time
- $\Longrightarrow$ Not every function is computable (e.g., the Halting Problem)
  - ... among the other math flaws 👇
  - `https://www.youtube.com/watch?v=HeQX2HjkcNo`

Church-Turing thesis

Any function that is effectively computable (i.e., computable by an algorithm) can be computed by a Turing machine

Turing Completeness

A system (e.g., a programming language or electronic circuitry) is said to be **Turing complete** if it can simulate a Turing machine.

Requirements for Turing completeness:

1. Conditional branching (e.g., `if...then...else`)
2. Memory that can grow "without bound" (e.g., arrays and lists).
3. The ability to loop or recurse (i.e., repeat steps indefinitely)

Turing complete and NOT complete examples

NOT Turing complete:

- Hypertext Markup Language (HTML): only describes structure
- ANSI SQL: declarative query language
  - Lacks features like looping/recursion

Turing complete

- Computers!!!
- Conway's Game of Life [Ren11]
- Magic: The Gathering [CBH19]
- Minecraft Redstone Circuits [HFP+21]
- PowerPoint 😂
  - As long as someone is clicking a button to iterate the slides!

Universal Turing Machine

Alan Turing's brilliant 1936 extension [Tur36]

- **Turing Machine** (TM) solves *one fixed problem*
  - Its transition rules are hardwired
- **Universal Turing Machine** (UTM) can simulate *any other* TM
- It does this by reading as input:
  - An encoding (i.e., description) of another Turing machine M
  - An input w for that machine
  - The UTM then behaves as if it were M running on w
- $\Longrightarrow$ Programs themselves can be treated as *data*

Modern computers are real-world UTMs (but with finite memory)

This is why we can run "computers on our computers" as emulators and virtual machines

Outline

Von Neumann architecture (1945)

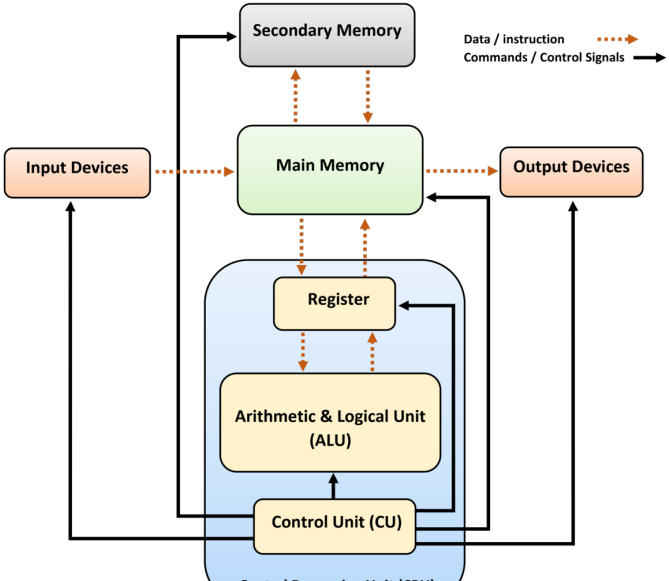
A *finite approximation* of a Universal Turing Machine

- Memory: holds both data and instructions
- Central Processing Unit (CPU): executes the instructions step by step
- Control Unit: decides what instruction to execute next
- Input/Output devices: handle communication with the outside world

But also... Harvard architecture (1944)

- Instructions and data are stored in separate memory units
- Many advantages (i.e., speed/optimizations and security/reliability)
- Dealbreaker: more complex $\implies$ more costly
- Used in modern microcontrollers and embedded systems

Modern Computer Architecture (based on Von Neumann)



32-bit vs. 64-bit Architecture

It is the register width and impacts:

- Addressable memory
  - 32-bit: up to 2^{32} bytes $= 4$ GiB of RAM
  - 64-bit: up to 2^{64} bytes ≈ 16 exabytes of RAM
    - Practically limited by physical space
- Data Bus Width
  - 64-bit bus can transfer twice as much data per clock cycle
- Instructions
  - 64-bit mode introduces additional general-purpose registers
  - Programs compiled for 64-bit CPUs are bigger than the ones for 32-bit

Nowadays, 32-bit architectures are only used for lightweight embedded systems or still exist in old devices.

CISC and RISC

Complex Instruction Set Computer (CISC)

- Instructions are capable of multi-step operations or complex addressing modes
- Require several cycles per instruction
- e.g., **x86**

Reduced Instruction Set Computer (RISC)

- Small and highly-optimized set of instructions
- Lower cycles per instruction than CISC
- Memory is only accessed through specific instructions
  - Rather than most instructions as in CISC
- e.g., **ARM**

The CISC approach attempts to minimize the number of instructions per program, sacrificing the number of cycles per instruction.
RISC does the opposite, reducing the cycles per instruction at the cost of the number of instructions per program.

CISC vs. RISC

CISC

- "Many-cycles" complex instructions
- Small code size, but high cycles
- Memory-to-memory $\Rightarrow$ LOAD/STORE incorporated in instructions
- Transistors used for implementing complex instructions

RISC

- "Few-cycles" simple instructions
- Low cycles, but large code size
- Register-to-register $\Rightarrow$ LOAD/STORE are independent instructions
- Spends more transistors on memory registers

RISC beats CISC in performance $\times$ watt $\Longrightarrow$
RISC dominate battery-powered devices market

E.g., LOAD/STORE

Let's consider the following C statement: "`++x;`"
`x` is a global variable

- x86

```
inc [rax]       @ adds 1 to the destination operand
```

- ARM

```
ldr r0, =x      @ load &x into r0
ldr r1, [r0]    @ read the value of x
add r1, r1, #1  @ increment it
str r1, [r0]    @ write the result back
```

References I

[CBH19] Alex Churchill, Stella Biderman, and Austin Herrick.
Magic: The gathering is turing complete.
arXiv preprint arXiv:1904.09828, 2019.

[HFP+21] Stefan Höltgen, Thomas Fecker, Simon Pleikies, Natalie Wormsbecher, and Silvio Divani.
A case of toy computing implementing digital logics with "minecraft".
Andrew Adamatzky (Hg.): Alternative Computing. Heidelberg et al.: Springer, 2021.

[Ren11] Paul Rendell.
A universal turing machine in conway's game of life.
In *2011 International Conference on High Performance Computing & Simulation*, pages 764–772. IEEE, 2011.

References II

[Tur36] Alan M. Turing.
On computable numbers, with an application to the entscheidungsproblem.
Proceedings of the London Mathematical Society, 42:230–265, 1936.
Reprinted in M. Davis (Ed.), *The Undecidable*, Raven Press, 1965.